

React JS Series

Complete Study Notes: Concepts, Flows and Projects

Topics 01 to 34 are covered below. Each **main heading starts with its number** and has its own color. Dark boxes are code, tinted boxes are diagrams, yellow boxes are key points to remember.

01	Why React? The problem it solved	18	Appwrite backend
02	Create React projects	19	ENV and Appwrite setup
03	React flow and structure (Components)	20	Authentication service
04	Create your own React library and JSX	21	Database, file upload, queries
05	Why you need Hooks	22	Redux Toolkit in big projects
06	Virtual DOM, Fiber and Reconciliation	23	Production-grade components
07	Tailwind and Props	24	React Hook Form
08	Interview question: Counter	25	Form and slug values
09	Project: Background Changer	26	Building pages
10	useEffect, useRef and useCallback	27	CORS and debugging
11	Custom Hooks (Currency project)	28	Deploy to production
12	React Router	29	Assignments
13	Context API	30	Series ends
14	Context API with Local Storage	31	Connect frontend and backend
15	Redux Toolkit	32-33	Interview questions
16-17	Mega Project	34	Handle APIs like a pro

01

Why Need React? What Problem Did It Solve?

Short answer: React was created because building large, highly interactive web interfaces with traditional JavaScript became hard to manage. The core problem: as a UI becomes more dynamic, **keeping the DOM synchronized with changing data** gets very complex.

Old mindset	React mindset
"Something changed. Which DOM elements do I update manually?"	"My UI is a function of my data. If data changes, React figures out what to change."

1. Before React: the web was documents

Early websites were documents. The server generated HTML, the browser displayed it. Another page meant: Home, Server, new HTML, Browser. There was little client-side state.

2. JavaScript arrived

JavaScript made pages interactive. The DOM became something JS could manipulate without reloading.

```
<button onclick="changeText()">Click Me</button>
<p id="message">Hello</p>

function changeText() {
  document.getElementById("message").innerText = "Hello Chakresh";
}
```

3. Web apps became complicated

Apps like Facebook, Gmail, Google Maps, Twitter/X, dashboards, chat and e-commerce behave like desktop apps. One state change can touch many places:

```
User likes post
  |
  v
Like count changes -> Button appearance changes
  |
  v
Notification changes -> Activity feed may change
```

With plain JS you end up writing **getElementById**, **querySelector**, **innerHTML**, **classList.add/remove** everywhere, and must decide which part of the DOM to change each time.

4. The real problem was not JavaScript

The problem was **managing UI state and DOM updates at scale**. Manual updates (likes++ then update #likeCount) are fine in small apps but not with 10,000 interacting UI pieces.

5. Facebook: origin of React

- React originated at **Facebook (Meta)**. **Jordan Walke** built the early prototype.
- It evolved through the wider Facebook engineering effort on large dynamic UIs.

6-7. News Feed and the Cascading Update problem

Clicking Like on one post can affect the feed, notifications, profile, sidebar and counters. Developers maintained **Application State + DOM State** and had to keep them in sync. That is the difficult part.

■ 8. React's big idea

Instead of "change this DOM element", you describe: **"Given this state, this is what my UI should look like."**

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button>Count: {count}</button>;  
}
```

■ 9. Declarative vs Imperative UI

Imperative (how)	Declarative (what)
Find the element and change its text: document.querySelector("#counter").innerText = "Count: 5"	Describe the UI: <button>Count: {count}</button>. React handles the DOM update.

■ 10. React's mental model

STATE -> COMPONENT -> UI

State changes -> React re-renders -> React compares UI
-> React updates only the necessary DOM

■ 11. What is the Virtual DOM?

React keeps an in-memory JavaScript representation of the UI, compares old and new versions, then updates only what changed. Example: Count: 0 to Count: 1 changes only the text.

Modern clarification: Don't reduce React to "a Virtual DOM library". The bigger innovation is the **component + declarative rendering model**, with reconciliation handling updates.

■ 12. Components

UI is broken into reusable components instead of one pile of DOM code.

```
App  
|-- Navbar  
|-- Sidebar  
|-- PostList  
|   |-- Post  
|   |-- Post  
|   |-- Post  
|-- Footer
```

```
function Post({ title }) {  
  return (<article><h2>{title}</h2><button>Like</button></article>);  
}  
<Post title="Learning React" />  
<Post title="Learning JavaScript" />
```

■ 13. XHP at Facebook

Before React, Facebook built **XHP** to make HTML generation safer and more component-oriented on the server. Its ideas influenced React, which brought components to client-side JavaScript.

■ 14. JSX

JSX puts markup and logic together. Instead of splitting by file type (HTML / CSS / JS), React organizes by **component**: UI structure + logic + state together.

■ 15-16. History and early reaction

- Early work: **2011-2012** at Facebook. Public introduction: **JSConf US, May 2013** (open-sourced the same year).
- Developers first reacted: "HTML inside JavaScript?!" The classic split was HTML = structure, CSS = style, JS = behavior. React proposed Component = structure + behavior + state.

■ 17. Why developers came to like it

Traditional thinking	React thinking
What changed? Which DOM element? What depends on it? Did I forget something? What if another event changes it?	What is my state? Given this state, what should my UI look like?

■ 18. Old DOM code vs React

```
// Old style
let count = 0;
function add() { count++; document.querySelector("#count").innerText = count; }

// React
function Counter() {
  const [count, setCount] = useState(0);
  return (<>
    <h1>{count}</h1>
    <button onClick={() => setCount(count + 1)}>Add</button>
    <button onClick={() => setCount(count - 1)}>Remove</button>
  </>);
}
```

■ 19. The real problem React solved

Remember: UI = f(state). The UI is a function of the current state. Remember this idea.

■ 20. React did not invent everything

```
Functional programming + Component-based UI + Declarative programming
+ Virtual DOM / reconciliation + Facebook's large-scale UI experience
      |
      v
    React
```

■ 21. Timeline

1990s : Web mostly documents -> JavaScript arrives -> DOM manipulation
2000s : AJAX, rich apps (Gmail, Maps, social networks)
2010+ : Very interactive apps, DOM sync becomes difficult
Facebook : Large-scale UI problems, XHP, Jordan Walke builds React
2011-12 : Early React development
2013 : React publicly introduced and open-sourced
Today : Components, Hooks, state management, SSR, full-stack frameworks

■ 22. So why do YOU need React?

LARGE UI -> MANY COMPONENTS -> MANY STATES -> STATE CHANGES
-> UI MUST STAY SYNCED -> DOM MANIPULATION GETS HARD -> REACT
(Components | Declarative UI | State-driven rendering)

Don't learn `useState`, `props`, `hooks` and `JSX` as isolated syntax. Understand the problem first. **`useState`** is not just a fancy variable: when state changes, React knows, re-renders the component, finds UI changes and updates the DOM.

02 Create React Projects

- **Vite** is a bundler: it bundles code and assets, and is fast and efficient for modern web development.
- **react** = core library, **react-dom** = web pages, **react-native** = mobile apps.
- **npm / npx** = Node package manager. **create-react-app** (`npx create-react-app`) is the old method.
- **package.json** holds scripts to run/build, plus versions of vite, react, etc.

```
npm create vite@latest
npm install           # creates node_modules
npm run dev
```

■ Why learn React JS?

- Makes it easy to manage and build complex front ends; ideal for **Single Page Applications**.
- It is a **library**, not a strict framework: use any state, routing or styling library; no forced folder structure.
- Doesn't reload the whole page; uses a copy of the real DOM (Virtual DOM) and reuses components.
- It follows convention, not strict rules.

■ Topics to learn

- Core: state / UI manipulation, `JSX`
- Reusing components (`props`)
- Propagating change (`hooks`)
- SPA
- Internals: Virtual DOM, Fiber, diffing algorithm, hydration

■ Add-ons (React doesn't include these)

- **Router** (React has none) and **state management**: Redux, Redux Toolkit, Zustand, Recoil, Jotai, MobX, Valtio, XState, Context API.
- Class components (legacy code).
- BaaS apps (social media / e-commerce clones): Firebase, Supabase, Appwrite, PocketBase.

■ After React

React alone is not a complete solution in most cases (no SEO, browser renders JS, no routing). Frameworks: **Next.js**, **Gatsby**, **Remix**, **Qwik**, **SolidJS**, **Astro**, **SvelteKit**, **Nuxt**, **Angular**, **Vue**.

03 React Flow, Structure and Components

Definition: A component is a **small, reusable building block** of a React app: reusable UI plus the logic and structure to display part of your application.

Think of a house: House has Door, Window, Kitchen, Bedroom. Similarly App has Navbar, Sidebar, ProductList (ProductCards) and Footer.

■ 1. First component

```
function Welcome() { return <h1>Hello Chakresh</h1>; }  
function App() { return (<div><Welcome /></div>); }
```

■ 2. Why components?

Without them you get one huge App.jsx with Navbar, Product, Cart, Login and Footer code. With them:

```
src/  
|-- App.jsx  
|-- components/  
    |-- Navbar.jsx    |-- ProductCard.jsx    |-- Cart.jsx  
    |-- Login.jsx     |-- Footer.jsx
```

■ 3-4. Reusable, and props make them different

```
function ProductCard({ name, price }) {  
  return (<div><h2>{name}</h2><p>{price}</p></div>);  
}  
<ProductCard name="iPhone" price="Rs 70,000" />  
<ProductCard name="Samsung" price="Rs 50,000" />  
<ProductCard name="OnePlus" price="Rs 40,000" />
```

■ 5. Component + Props + State

- **UI:** what it returns.
- **Props:** data from the parent, e.g. `<Product name="iPhone" />`.
- **State:** data the component remembers, e.g. `const [count, setCount] = useState(0)`.

■ 6. A component is a function that returns UI

A functional component is a JavaScript function that returns JSX describing UI.

■ 7. Naming rule

Rule: Component names **must start with a capital letter** (Header, ProductCard). React uses capitalization to tell components (`<Header />`) from HTML elements (`<div />`).

■ 8. Mental model

A component is an independent, reusable piece of UI that can receive data, maintain state and describe how its UI should look. Core idea: **STATE, COMPONENT, UI**.

04 Create Your Own React Library and JSX

Purpose: understand what React does behind the scenes. JSX (HTML-like syntax) is converted into a JavaScript object, then rendered in the browser.

JSX -> JavaScript object -> React renderer -> Real DOM

```
<h1>Hello</h1> // JSX
createElement("h1", null, "Hello") // roughly converted to
{ type: "h1", props: { children: ["Hello"] } } // object returned
```

- **createElement()** describes the UI.
- **render()** creates the DOM (e.g. document.createElement("h1")).

05 Why You Need Hooks (and Project)

Hooks let functional components use React features such as state and lifecycle behavior. Before Hooks, state mostly required class components, which were more complicated.

```
// Before: class Counter extends React.Component { state = { count: 0 }; }
// Now:
function Counter() {
  const [count, setCount] = useState(0);
  return <h1>{count}</h1>;
}
```

Need	Hook
State	useState()
Side effects	useEffect()
Shared context	useContext()
Reusable logic	Custom Hooks

Remember: A Hook is not just a normal JS function. **useState()** tells React: "Remember this value for this component and give me a way to update it."

06 Virtual DOM, Fiber and Reconciliation

Three connected concepts that explain how React updates the UI efficiently.

1. Virtual DOM

A JavaScript representation of the UI, e.g. { type: "h1", children: ["Hello"] }. When state changes, a new representation is built.

2. Reconciliation

React compares the old and new UI trees to find what changed. Example: Old <h1>Hello</h1> vs New <h1>Hello Chakresh</h1>: the tag is the same, only text changed, so only that part of the DOM updates.

3. Fiber

React's internal architecture for doing this work. It splits rendering into small units called **Fibers**.

- Organizes rendering work
- Prioritizes updates

- Can pause / resume rendering
- Handles large component trees efficiently

07 Tailwind and Props in React JS

1. Props

Props = data passed from **parent to child**; think of them as arguments to a component.

```
function User({ name }) { return <h2>Hello {name}</h2>; }
function App() { return <User name="Chakresh" />; }
```

2. Tailwind CSS

Tailwind = utility CSS classes written directly in className.

```
<h1 className="text-3xl font-bold text-blue-600">Hello Chakresh</h1>
```

Class	Meaning
text-3xl	font size
font-bold	font weight
text-blue-600	text color

3. Props + Tailwind together

```
function Button({ text }) {
  return <button className="bg-blue-500 text-white px-4 py-2 rounded">{text}</button>;
}
<Button text="Login" />   <Button text="Sign Up" />
```

Remember: Props = data, Tailwind = styling, Components = reusable UI.

08 Interview Question: Counter

Requirements:

- Initial value 0 (the lecture code starts at 15).
- Add increments; Remove decrements.
- Never below 0 and never above 20.
- Show a message when the user tries to go out of range.

```
function App() {
  let [counter, setCounter] = useState(15)

  const addValue = () => {
    // setCounter((prevCounter) => prevCounter + 1)    <- interview question
    // setCounter((prevCounter) => prevCounter + 1)
    if (counter < 20) {
      setCounter(counter + 1)
    } else {
      console.log("Not greater than 20")
    }
  }
}
```

Interview tip: Calling `setCounter(counter + 1)` twice only adds 1 (same stale value). Using the **updater form** (`prev => prev + 1`) twice adds 2.

09 Building a React Project: Background Changer

Buttons call `setColor("red")` etc., and the page background follows the color state.

```
<button onClick={() => setColor("red")}
  className="outline-none w-16 h-10 rounded-full text-white shadow-lg"
  style={{background: "red"}}>Red</button>

<button onClick={() => setColor("green")}
  className="outline-none w-16 h-10 rounded-full text-white shadow-lg"
  style={{background: "green"}}>Green</button>
```

10 useEffect, useRef and useCallback

Hook	Purpose	Syntax
useEffect	Run side effects ("do something after render")	useEffect(callback, dependencies)
useRef	Store a value / access DOM without re-render ("remember something")	useRef(initialValue)
useCallback	Cache a function between re-renders ("remember a function")	useCallback(fn, dependencies)

11 Custom Hooks (Currency Project)

A **Custom Hook** is a reusable function whose name starts with **use** and which can call other Hooks to share **stateful logic** between components.

- Example: two components need the same counter logic. Instead of copying it, create a Custom Hook.
- A Custom Hook shares **logic**, not UI.

12 React Router Crash Course

React Router is a library for **client-side routing**: it moves between views/components based on the URL **without a full page reload**.

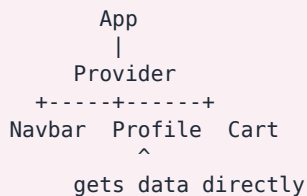
13 Context API Crash Course

Context API shares data between components **without passing props through every level**.

■ The problem: prop drilling

App -> (props) Navbar -> (props) UserMenu -> (props) Profile
User data passes through components that don't need it.

■ Context solves it



```
const UserContext = createContext();

function App() {
  return (
    <UserContext.Provider value="Chakresh">
      <Profile />
    </UserContext.Provider>
  );
}

function Profile() {
  const user = useContext(UserContext);
  return <h1>Hello {user}</h1>;
}
```

- Common uses: logged-in user, theme (dark/light), language, app settings, shared state.
- **Props**: Parent to Child. **Context**: Provider to any descendant.
- React.createContext() returns a context object; a subscribing component reads the value from the **closest matching Provider** above it.

14 Context API with Local Storage

Tool	Solves
Context API	Shares data between components
Local Storage	Saves data in the browser so it survives refresh

Together they suit theme, login preferences, cart data, etc. Example project: **Dark / Light theme**.

15 Redux Toolkit Crash Course

Redux Toolkit (RTK) is the official recommended way to use Redux for global state in larger apps. It provides APIs like **configureStore** and **createSlice**.

```
Navbar --+
Cart ----> Global State (central store)
Profile --+
Checkout +

Component -> dispatch(action) -> Reducer -> Store changes -> Component gets updated state
```

16-1
7

Mega Project in React

Choosing the mega project (16) and building it "the hard way" (17). The project built in this series is a **blog app: Login and Read Post web page** using Appwrite as backend.

18

Appwrite Backend for React Project

- **Appwrite** = web service acting as the back end (sign up, create a project, create a user).
- **TinyMCE**: React rich-text editor component.
- **html-react-parser**: renders HTML strings as React elements (raw HTML can't be rendered directly).
- **React Hook Form**: manages inputs and avoids input-handling errors.

19

ENV and Appwrite in React Project

```
npm i @reduxjs/toolkit react-redux react-router-dom appwrite
npm i @tinymce/tinymce-react html-react-parser react-hook-form
```

Package	Purpose
@reduxjs/toolkit	Global state (logged-in user, posts) with less boilerplate
react-redux	Connects Redux to React (Provider, useSelector, useDispatch)
react-router-dom	Multiple routes (/login, /add-post) without reload
appwrite	SDK for auth, database and file storage, no custom server needed
@tinymce/tinymce-react	Rich text editor for post content
html-react-parser	Turns TinyMCE's HTML string into React elements
react-hook-form	Form inputs, validation, submission with fewer re-renders

Updated: Appwrite naming update: **Collections to Tables, Attributes to Columns, Documents to Rows.**

Keep secrets in a **.env** file. Vite reads **import.meta.env.VITE_APPWRITE_URL** (CRA used **process.env.REACT_APP_APPWRITE_URL**).

20 Build Authentication Service with Appwrite

- Folder **conf/conf.js** for configuration; folder **appwrite/auth.js** for the auth service.
- Methods: **createAccount**, **login**, **getCurrentUser**, **logout**.
- Docs: appwrite.io/docs/products/auth/email-password

21 Appwrite Database, File Upload and Custom Queries

Create **appwrite/config.js** with **export class Service** and a constructor.

Group	Methods
CRUD	createPost, updatePost, deletePost, getPost
File system	uploadFile, deleteFile, getFilePreview

22 Configure Redux Toolkit in Big Projects

```
store/          -> authSlice.js, store.js
components/     -> Header/Header.jsx, Footer/Footer.jsx, index.js
Also update App.jsx and main.jsx
```

23 Production-Grade React Components

- **container/Container.jsx**, completed **Footer.jsx**, new **Logo.jsx**.
- **Header/LogoutBtn.jsx**, updated **Header.jsx**, **Button.jsx**.
- Introduction to **useRef** with **forwardRef** (forward reference hook) in **Input.jsx**.

24 React Hook Form in Production

- Files: **Select.jsx**, **PostCard.jsx**, **Login.jsx** (React Hook Form), **Signup.jsx**, **AuthLayout.jsx**.

25 Adding Form and Slug Values

- **RTE.jsx** (rich text editor) using React Hook Form's register fields:
react-hook-form.com/get-started#Registerfields
- Folder **components/post-form** with file **PostForm.jsx**.

26 Building Pages

Folder **pages**: Signup.jsx, Login.jsx, AddPost.jsx, AllPosts.jsx, EditPost.jsx, Home.jsx, Post.jsx.
Update **main.jsx** with the routes.

27 CORS and Debugging in React Project

A CORS error from Appwrite means it doesn't recognize your app's address as an allowed platform.

The fix is in the Appwrite console, not your code.

- Open the Appwrite console and select your project.
- In Overview, scroll to **Integrations, then Platforms** (older: Overview, Add a platform).
- Click **Add platform** and choose **Web**.
- Enter a name (e.g. local) and hostname **localhost** (no http://, port or path).
- Save, then hard-refresh (**Ctrl + Shift + R**).

Remember: When you deploy, add another Web platform with your live domain (e.g. myblog.vercel.app).

28 Deploy React App to Production

- **Vercel** and **Netlify** are platforms that host your application on their servers.
- Deployed project: <https://logincreatepostappproject.vercel.app/>

29 Assignments for Your React Course

Frontend: React app. Backend: Appwrite. Build:

Frontend skills	Backend (Appwrite)
Signup, Login, Logout Redux / Context Routing CRUD (text) File handling	Auth Database File management

30 React Series Ends Here

Next topics covered below: connecting front end and back end, interview questions and professional API handling.

31 Connect Frontend and Backend in JavaScript (Proxy and CORS)

1. **Build the backend API.** Express routes like GET /api/users with app.use(express.json()) on port 3000.
2. **Build the frontend** on its own dev server (e.g. Vite on 5173).
3. **Call the API** using fetch("/api/users").
4. **Handle the cross-origin problem** (different ports = different origins). Option A, Proxy (recommended for dev): add a proxy for /api to http://localhost:3000 in vite.config.js. Option B, CORS: npm install cors, then app.use(cors({ origin: "http://localhost:5173" }))) before routes.
5. **If using cookies:** credentials: true in CORS config and credentials: "include" in fetch; use an exact origin, not *.
6. **Use an env variable** for the API URL: leave VITE_API_URL unset in dev (proxy), set it to the real API domain in production.
7. **Test:** DevTools Network tab should show 200 with no CORS errors.
8. **Deploy:** serve the built frontend from Express (same origin), or Nginx routing / to static files and /api to the backend, or separate domains with CORS limited to your frontend origin.
9. **Verify in production:** HTTPS, cookie settings (SameSite=None; Secure for cross-site), and the CORS allow-list matching the real frontend URL.

32-3 3

Interview Questions

- **32.** "You don't need state for this": a React interview question (derive values from existing state or props instead of adding new state).
- **33.** Testing your data-type skills in React interviews.

34

Handle APIs Like a Pro in React (Custom Query, Axios, Race Condition)

Component -> Hook -> Axios -> Backend
Request interceptor : adds token
Response interceptor: handles 401, errors, retries (GET only)
Hook updates data / loading / error -> Component re-renders

- **Race condition fix:** a new request cancels the old one.
- **Search:** debounce input, then send one request.
- **Mutations:** disable the button while loading, then invalidate cache and refetch.
- **Scale up:** use TanStack Query.